

JAVA EXCEPTION HANDLING

10 Real-Application Programs

|

Roll Number: A25126552251

Name: S.Trivikram

1. ArrayIndexOutOfBoundsException

Real Application: Student Marks System

1. Question: Write a Java program to demonstrate **ArrayIndexOutOfBoundsException** using student marks system.

2. Aim: To develop a Java program that handles **ArrayIndexOutOfBoundsException** using try-catch in a real-world application.

3. Algorithm:

1. Start the program.
2. Create the required real-world data or object.
3. Place the operation that may cause the exception inside a try block.
4. If ArrayIndexOutOfBoundsException occurs, control moves to the catch block.
5. Display a meaningful error message.
6. Continue/terminate the program safely.
7. Stop.

4. Step-wise Execution:

1. Compile the Java source code.
2. Run the program.
3. The application performs the required operation.
4. The invalid condition produces ArrayIndexOutOfBoundsException.
5. The catch block catches the exception.
6. The error message is displayed without an unhandled exception.

5. UML Diagram:

Main Class	Application / Method	Exception Handler
Student Marks	try → operation	catch (ArrayIndexOutOfBoundsException) → error message

6. Java Source Code:

```
class StudentMarks {  
    public static void main(String[] args) {  
        int[] marks = {80, 75, 90};  
  
        try {  
            System.out.println("Student Mark = " + marks[5]);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Error: Invalid mark position.");  
        }  
    }  
}
```

7. Output of the Program:

Error: Invalid mark position.

8. Result: Thus, the Java program successfully demonstrates **ArrayIndexOutOfBoundsException** and handles the exception using try-catch.

2. NullPointerException

Real Application: Customer Details System

1. Question: Write a Java program to demonstrate **NullPointerException** using customer details system.

2. Aim: To develop a Java program that handles **NullPointerException** using try-catch in a real-world application.

3. Algorithm:

1. Start the program.
2. Create the required real-world data or object.
3. Place the operation that may cause the exception inside a try block.
4. If **NullPointerException** occurs, control moves to the catch block.
5. Display a meaningful error message.
6. Continue/terminate the program safely.
7. Stop.

4. Step-wise Execution:

1. Compile the Java source code.
2. Run the program.
3. The application performs the required operation.
4. The invalid condition produces **NullPointerException**.
5. The catch block catches the exception.
6. The error message is displayed without an unhandled exception.

5. UML Diagram:

Main Class	Application / Method	Exception Handler
Customer Details	try → operation	catch (NullPointerException) → error message

6. Java Source Code:

```
class Customer {
    public static void main(String[] args) {
        String customerName = null;

        try {
            System.out.println("Customer Name Length = "
                               + customerName.length());
        } catch (NullPointerException e) {
            System.out.println("Error: Customer name is not available.");
        }
    }
}
```

7. Output of the Program:

Error: Customer name is not available.

8. Result: Thus, the Java program successfully demonstrates **NullPointerException** and handles the exception using try-catch.

3. NumberFormatException

Real Application: Bank Account System

1. Question: Write a Java program to demonstrate **NumberFormatException** using bank account system.

2. Aim: To develop a Java program that handles **NumberFormatException** using try-catch in a real-world application.

3. Algorithm:

1. Start the program.
2. Create the required real-world data or object.
3. Place the operation that may cause the exception inside a try block.
4. If **NumberFormatException** occurs, control moves to the catch block.
5. Display a meaningful error message.
6. Continue/terminate the program safely.
7. Stop.

4. Step-wise Execution:

1. Compile the Java source code.
2. Run the program.
3. The application performs the required operation.
4. The invalid condition produces **NumberFormatException**.
5. The catch block catches the exception.
6. The error message is displayed without an unhandled exception.

5. UML Diagram:

Main Class	Application / Method	Exception Handler
Bank Account	try → operation	catch (NumberFormatException) → error message

6. Java Source Code:

```
class BankAccount {  
    public static void main(String[] args) {  
        String balance = "ABC";  
  
        try {  
            int amount = Integer.parseInt(balance);  
            System.out.println("Balance = " + amount);  
        } catch (NumberFormatException e) {  
            System.out.println("Error: Invalid balance format.");  
        }  
    }  
}
```

7. Output of the Program:

Error: Invalid balance format.

8. Result: Thus, the Java program successfully demonstrates **NumberFormatException** and handles the exception using try-catch.

4. StringIndexOutOfBoundsException

Real Application: Mobile Number Processing

1. Question: Write a Java program to demonstrate **StringIndexOutOfBoundsException** using mobile number processing.

2. Aim: To develop a Java program that handles **StringIndexOutOfBoundsException** using try-catch in a real-world application.

3. Algorithm:

1. Start the program.
2. Create the required real-world data or object.
3. Place the operation that may cause the exception inside a try block.
4. If StringIndexOutOfBoundsException occurs, control moves to the catch block.
5. Display a meaningful error message.
6. Continue/terminate the program safely.
7. Stop.

4. Step-wise Execution:

1. Compile the Java source code.
2. Run the program.
3. The application performs the required operation.
4. The invalid condition produces StringIndexOutOfBoundsException.
5. The catch block catches the exception.
6. The error message is displayed without an unhandled exception.

5. UML Diagram:

Main Class	Application / Method	Exception Handler
Mobile Number Processing	try → operation	catch (StringIndexOutOfBoundsException) → error message

6. Java Source Code:

```
class MobileNumber {  
    public static void main(String[] args) {  
        String mobile = "9876543210";  
  
        try {  
            System.out.println("Digit = " + mobile.charAt(15));  
        } catch (StringIndexOutOfBoundsException e) {  
            System.out.println("Error: Invalid mobile number position.");  
        }  
    }  
}
```

7. Output of the Program:

Error: Invalid mobile number position.

8. Result: Thus, the Java program successfully demonstrates **StringIndexOutOfBoundsException** and handles the exception using try-catch.

5. ClassCastException

Real Application: Employee Management System

1. Question: Write a Java program to demonstrate **ClassCastException** using employee management system.

2. Aim: To develop a Java program that handles **ClassCastException** using try-catch in a real-world application.

3. Algorithm:

1. Start the program.
2. Create the required real-world data or object.
3. Place the operation that may cause the exception inside a try block.
4. If ClassCastException occurs, control moves to the catch block.
5. Display a meaningful error message.
6. Continue/terminate the program safely.
7. Stop.

4. Step-wise Execution:

1. Compile the Java source code.
2. Run the program.
3. The application performs the required operation.
4. The invalid condition produces ClassCastException.
5. The catch block catches the exception.
6. The error message is displayed without an unhandled exception.

5. UML Diagram:

Main Class	Application / Method	Exception Handler
Employee Management	try → operation	catch (ClassCastException) → error message

6. Java Source Code:

```
class Employee { }

class Manager extends Employee { }

class Developer extends Employee { }

class EmployeeSystem {
    public static void main(String[] args) {
        Employee emp = new Developer();

        try {
            Manager manager = (Manager) emp;
            System.out.println("Employee is a Manager.");
        } catch (ClassCastException e) {
            System.out.println("Error: Employee is not a Manager.");
        }
    }
}
```

7. Output of the Program:

Error: Employee is not a Manager.

8. Result: Thus, the Java program successfully demonstrates **ClassCastException** and handles the exception using try-catch.

6. **IllegalArgumentException**

Real Application: Driving License Validation

1. Question: Write a Java program to demonstrate **IllegalArgumentException** using driving license validation.

2. Aim: To develop a Java program that handles **IllegalArgumentException** using try-catch in a real-world application.

3. Algorithm:

1. Start the program.
2. Create the required real-world data or object.
3. Place the operation that may cause the exception inside a try block.
4. If **IllegalArgumentException** occurs, control moves to the catch block.
5. Display a meaningful error message.
6. Continue/terminate the program safely.
7. Stop.

4. Step-wise Execution:

1. Compile the Java source code.
2. Run the program.
3. The application performs the required operation.
4. The invalid condition produces **IllegalArgumentException**.
5. The catch block catches the exception.
6. The error message is displayed without an unhandled exception.

5. UML Diagram:

Main Class	Application / Method	Exception Handler
Driving License Validation	try → operation	catch (IllegalArgumentException) → error message

6. Java Source Code:

```
class DrivingLicense {  
    static void checkAge(int age) {  
        if (age < 18) {  
            throw new IllegalArgumentException(  
                "Age must be 18 or above.");  
        }  
        System.out.println("Eligible for driving license.");  
    }  
  
    public static void main(String[] args) {  
        try {  
            checkAge(16);  
        } catch (IllegalArgumentException e) {  
            System.out.println("Error: " + e.getMessage());  
        }  
    }  
}
```

7. Output of the Program:

Error: Age must be 18 or above.

8. Result: Thus, the Java program successfully demonstrates **IllegalArgumentException** and handles the exception using try-catch.

7. FileNotFoundException

Real Application: Bank Statement System

1. Question: Write a Java program to demonstrate **FileNotFoundException** using bank statement system.

2. Aim: To develop a Java program that handles **FileNotFoundException** using try-catch in a real-world application.

3. Algorithm:

1. Start the program.
2. Create the required real-world data or object.
3. Place the operation that may cause the exception inside a try block.
4. If **FileNotFoundException** occurs, control moves to the catch block.
5. Display a meaningful error message.
6. Continue/terminate the program safely.
7. Stop.

4. Step-wise Execution:

1. Compile the Java source code.
2. Run the program.
3. The application performs the required operation.
4. The invalid condition produces **FileNotFoundException**.
5. The catch block catches the exception.
6. The error message is displayed without an unhandled exception.

5. UML Diagram:

Main Class	Application / Method	Exception Handler
Bank Statement	try → operation	catch (FileNotFoundException) → error message

6. Java Source Code:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

class BankStatement {
    public static void main(String[] args) {
        try {
            File file = new File("bankstatement.txt");
            Scanner sc = new Scanner(file);

            System.out.println("Bank statement opened successfully.");
            sc.close();
        } catch (FileNotFoundException e) {
            System.out.println("Error: Bank statement file not found.");
        }
    }
}
```

7. Output of the Program:

Error: Bank statement file not found.

8. Result: Thus, the Java program successfully demonstrates **FileNotFoundException** and handles the exception using try-catch.

8. IOException

Real Application: Employee File System

1. Question: Write a Java program to demonstrate **IOException** using employee file system.

2. Aim: To develop a Java program that handles **IOException** using try-catch in a real-world application.

3. Algorithm:

1. Start the program.
2. Create the required real-world data or object.
3. Place the operation that may cause the exception inside a try block.
4. If IOException occurs, control moves to the catch block.
5. Display a meaningful error message.
6. Continue/terminate the program safely.
7. Stop.

4. Step-wise Execution:

1. Compile the Java source code.
2. Run the program.
3. The application performs the required operation.
4. The invalid condition produces IOException.
5. The catch block catches the exception.
6. The error message is displayed without an unhandled exception.

5. UML Diagram:

Main Class	Application / Method	Exception Handler
Employee File	try → operation	catch (IOException) → error message

6. Java Source Code:

```
import java.io.FileReader;
import java.io.IOException;

class EmployeeFile {
    public static void main(String[] args) {
        try {
            FileReader file = new FileReader("employee.txt");

            int data = file.read();
            System.out.println("Employee data read successfully.");

            file.close();
        } catch (IOException e) {
            System.out.println("Error: Unable to read employee file.");
        }
    }
}
```

7. Output of the Program:

Error: Unable to read employee file.

8. Result: Thus, the Java program successfully demonstrates **IOException** and handles the exception using try-catch.

9. SecurityException

Real Application: Company Security System

1. Question: Write a Java program to demonstrate **SecurityException** using company security system.

2. Aim: To develop a Java program that handles **SecurityException** using try-catch in a real-world application.

3. Algorithm:

1. Start the program.
2. Create the required real-world data or object.
3. Place the operation that may cause the exception inside a try block.
4. If SecurityException occurs, control moves to the catch block.
5. Display a meaningful error message.
6. Continue/terminate the program safely.
7. Stop.

4. Step-wise Execution:

1. Compile the Java source code.
2. Run the program.
3. The application performs the required operation.
4. The invalid condition produces SecurityException.
5. The catch block catches the exception.
6. The error message is displayed without an unhandled exception.

5. UML Diagram:

Main Class	Application / Method	Exception Handler
Company Security	try → operation	catch (SecurityException) → error message

6. Java Source Code:

```
class CompanySecurity {  
    public static void main(String[] args) {  
        try {  
            throw new SecurityException(  
                "Access denied to company resource.");  
        } catch (SecurityException e) {  
            System.out.println("Error: " + e.getMessage());  
        }  
    }  
}
```

7. Output of the Program:

Error: Access denied to company resource.

8. Result: Thus, the Java program successfully demonstrates **SecurityException** and handles the exception using try-catch.

10. IllegalStateException

Real Application: Online Shopping Order System

1. Question: Write a Java program to demonstrate **IllegalStateException** using online shopping order system.

2. Aim: To develop a Java program that handles **IllegalStateException** using try-catch in a real-world application.

3. Algorithm:

1. Start the program.
2. Create the required real-world data or object.
3. Place the operation that may cause the exception inside a try block.
4. If **IllegalStateException** occurs, control moves to the catch block.
5. Display a meaningful error message.
6. Continue/terminate the program safely.
7. Stop.

4. Step-wise Execution:

1. Compile the Java source code.
2. Run the program.
3. The application performs the required operation.
4. The invalid condition produces **IllegalStateException**.
5. The catch block catches the exception.
6. The error message is displayed without an unhandled exception.

5. UML Diagram:

Main Class	Application / Method	Exception Handler
Online Shopping Order	try → operation	catch (IllegalStateException) → error message

6. Java Source Code:

```
class OnlineOrder {  
    static boolean orderPlaced = false;  
  
    static void cancelOrder() {  
        if (!orderPlaced) {  
            throw new IllegalStateException(  
                "Order cannot be cancelled before placing.");  
        }  
        System.out.println("Order cancelled successfully.");  
    }  
  
    public static void main(String[] args) {  
        try {  
            cancelOrder();  
        } catch (IllegalStateException e) {  
            System.out.println("Error: " + e.getMessage());  
        }  
    }  
}
```

7. Output of the Program:

Error: Order cannot be cancelled before placing.

8. Result: Thus, the Java program successfully demonstrates **IllegalStateException** and handles the exception using try-catch.

All 10 Exceptions

No.	Exception	Application
1	ArrayIndexOutOfBoundsException	Student Marks
2	NullPointerException	Customer Details
3	NumberFormatException	Bank Account
4	StringIndexOutOfBoundsException	Mobile Number
5	ClassCastException	Employee Management
6	IllegalArgumentException	Driving License
7	FileNotFoundException	Bank Statement
8	IOException	Employee File
9	SecurityException	Company Security
10	IllegalStateException	Online Order